



CIN0136

Desenvolvimento de Software

Desenvolvimento de Software

Prof. Vinicius Cardoso Garcia

vcg@cin.ufpe.br :: @vinicius3w :: assertlab.com

<http://viniciusgarcia.me>



**Centro de
Informática**
UFPE



UNIVERSIDADE
FEDERAL
DE PERNAMBUCO

Licença do material

Este Trabalho foi licenciado com uma Licença



**Atribuição-NãoComercial-
Compartilha Igual 4.0 Internacional
(CC BY-NC-SA 4.0)**

Mais informações visite

<https://creativecommons.org/licenses/by-nc-sa/4.0/deed.pt> BR



Engenharia de Software em Dimensões

É um material abrangente, didático e atualizado que apresenta a Engenharia de Software como uma disciplina multidimensional. Cada capítulo foca em uma dimensão prática, conectando teoria e realidade profissional com clareza, profundidade e propósito.

Prof. Vinicius Cardoso Garcia

<https://bit.ly/esdbook>

Engenharia de Software em Dimensões

Da base conceitual à prática profissional



Vinicius Cardoso Garcia



Ponto de Commit

O podcast figital da disciplina (IF977) Engenharia de Software do curso de bacharelado em Sistemas de Informação do Centro de Informática (CIn) da Universidade Federal de Pernambuco (UFPE).

<https://go.uaite.xyz/ponto-de-commit>

Ponto de Commit



O podcast figital da disciplina de Engenharia de Software



Engenharia de Software Moderna

Engenharia de Software Moderna é um livro destinado a alunos de cursos de graduação em Computação. Ele pode ser lido também por profissionais que buscam um conhecimento horizontal da área.

Prof. Marco Tulio Valente

<https://engsoftmoderna.info>

@engsoftmoderna

ENGENHARIA — DE — SOFTWARE MODERNA

Princípios e práticas para
desenvolvimento de software
com produtividade



MARCO TULIO VALENTE





Code Review: Prática Profissional e Segurança

Revisitar fundamentos, aprimorar habilidades e entregar software com qualidade, responsabilidade e confiança.



O que vamos explorar hoje

01

Parte I — O Propósito

O valor real do code review além de encontrar bugs

03

Demonstração JS

Boas práticas aplicadas com ESLint, testes e código limpo

02

Parte II — Anatomia

Como conduzir uma revisão com profundidade e segurança

04

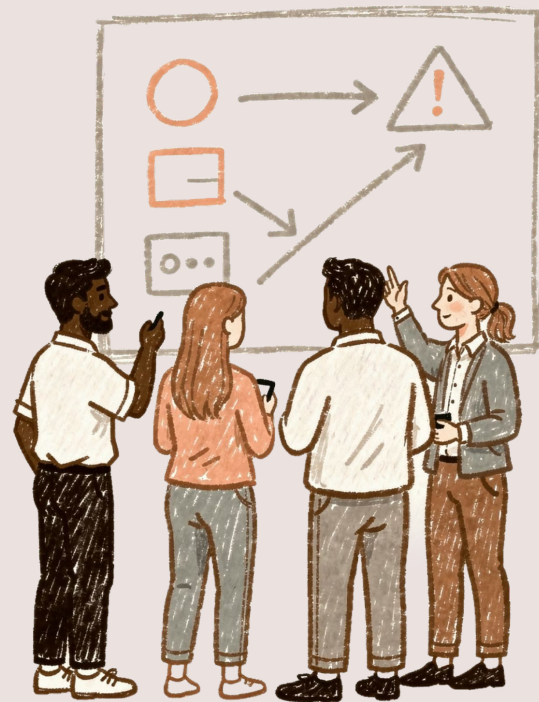
Parte III — Checklist

Critérios avançados para a entrega final do projeto



O Propósito da Revisão de Código

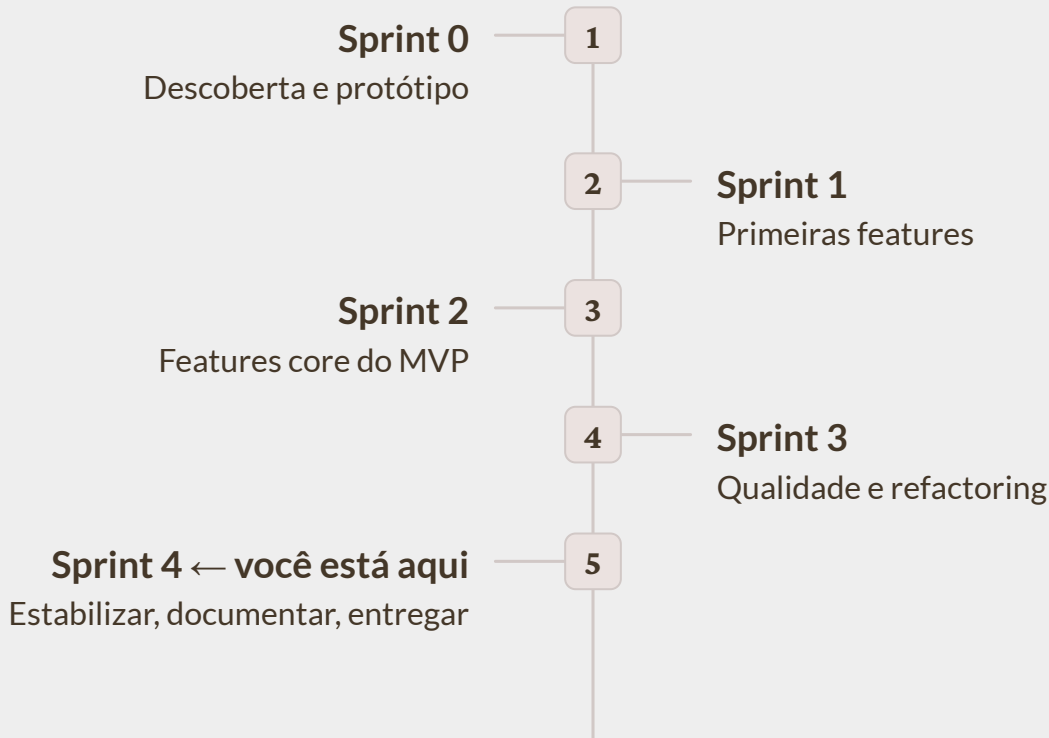
Code review não é apenas uma caçada a bugs, é um ritual de qualidade, aprendizado coletivo e proteção do produto.



Code Review como Prática Profissional

A entrega formal ao stakeholder está a **2 semanas**. Cada linha de código que entra agora precisa estar à altura do que você vai apresentar.

Você já passou por 4 sprints de desenvolvimento real. Já sentiu o PR travado esperando aprovação. Já aprovou um código que não entendia direito. Já recebeu um comentário que doeu. Esta aula não é sobre aprender code review do zero, é sobre entender **por que ele existe** e como fazê-lo de verdade.



Antes de Qualquer Coisa...

"Se outra equipe fosse assumir o repositório de vocês amanhã, o que eles diriam sobre o código que vocês escreveram?"

Pense de verdade. Não na resposta que você gostaria de dar, na resposta honesta. O código está legível? Os testes cobrem o que importa? As decisões arquiteturais estão documentadas ou só existem na cabeça de quem escreveu?

❏ Espaço para reflexão individual — 2 minutos antes de continuar.



Semana 6 → Semana 12

Você não é mais o mesmo desenvolvedor que viu code review pela primeira vez. Quatro sprints de código real mudam a perspectiva. Veja a diferença:

Dimensão	Semana 6	Semana 12
Pergunta central	"Isso funciona?"	"Isso pode durar?"
Foco do review	Erros e sintaxe	Funcionalidade + testes + legibilidade + arquitetura + docs
Experiência	Teoria recém-vista	4 sprints de código real no currículo

A maturidade não veio de uma aula. Veio de errar, corrigir, revisar e entregar com deadline real.



O Valor Real do Code Review



Encontrar Bugs

Identificar falhas lógicas antes que cheguem à produção



Compartilhar Conhecimento

Quebrar silos e distribuir entendimento do sistema



Segurança

A defesa final humana antes da produção



Formação

Acelerar o crescimento técnico de toda a equipe

Um bom revisor não corrige apenas o código, ele eleva o nível de toda a equipe.



A Última Linha de Defesa



Antes que qualquer mudança chegue ao usuário final, o code review é a **barreira humana mais crítica** do processo.

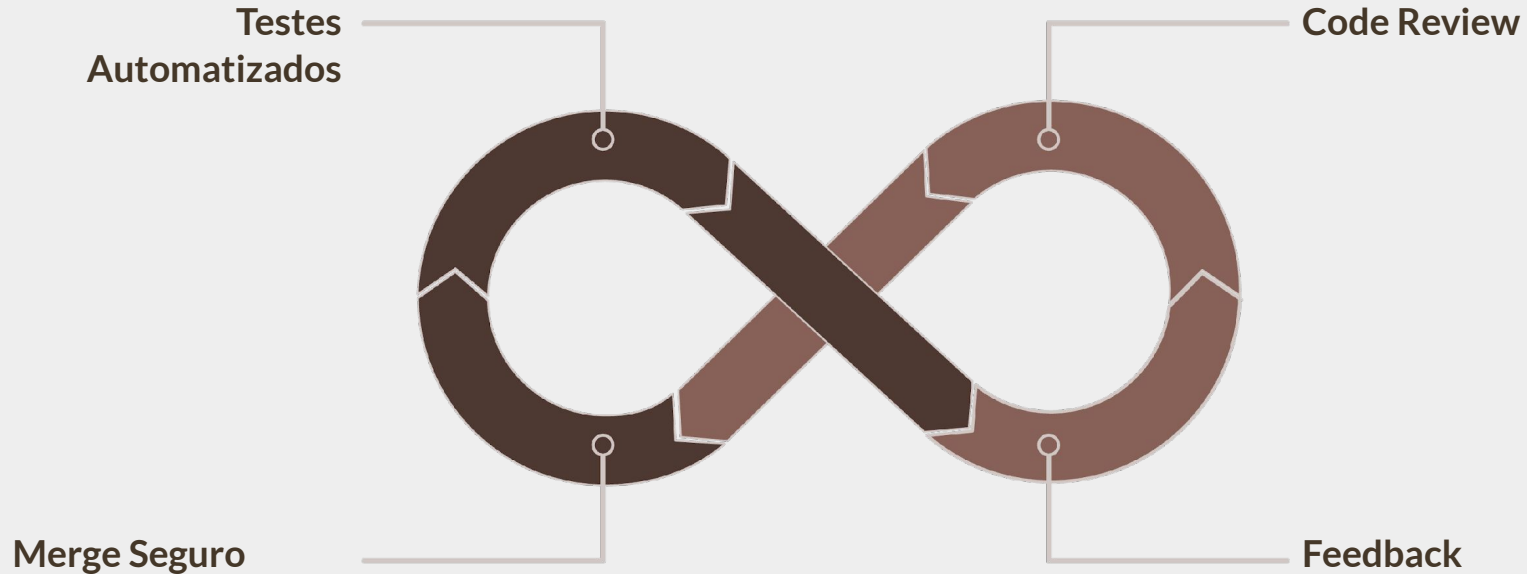


Um erro que passa pelo review pode custar horas de rollback, clientes insatisfeitos e débito técnico acumulado.

Cultivar uma cultura de revisão rigorosa é, antes de tudo, um ato de responsabilidade profissional.



O Ciclo da Qualidade



A automação (CI/CD, linters, cobertura de testes) deve atuar **antes** do olhar humano, liberando o revisor para o que realmente importa: julgamento, contexto e impacto.



Testes como Base de Confiança

Sem testes, o revisor precisa *imaginar* o comportamento do código. Com testes, ele pode **observar e verificar**.

- Testes unitários garantem contratos de função
- Testes de integração validam fluxos completos
- Cobertura mínima não é opcional, é critério de aceite

 Código sem teste é código não revisável com segurança.



Anatomia do Code Review

Cada revisão começa com uma pergunta fundamental: por que esta mudança é necessária?



Entendendo o Contexto

Qual problema resolve?

Leia o ticket, a descrição do PR e o histórico. O código deve refletir a intenção documentada.

Qual o impacto da mudança?

Alterações em módulos críticos exigem atenção redobrada a efeitos colaterais.

Qual o risco em produção?

Avaliar o custo de um erro, reputação, dados e disponibilidade do sistema.



O Papel do Revisor

✓ Análise Crítica

- Lógica e corretude
- Segurança e performance
- Aderência à arquitetura
- Cobertura de casos de erro

⚠ Não é Papel do Revisor

- Impor preferências de estilo pessoal
- Reescrever o código do autor
- Bloquear por detalhes sem impacto real



Quando e Como Intervir

1

Bloqueante

Bug crítico, vulnerabilidade de segurança ou violação grave de arquitetura

2

Sugestão

Melhoria de legibilidade, performance ou testabilidade, comentário construtivo

3

Nitpick

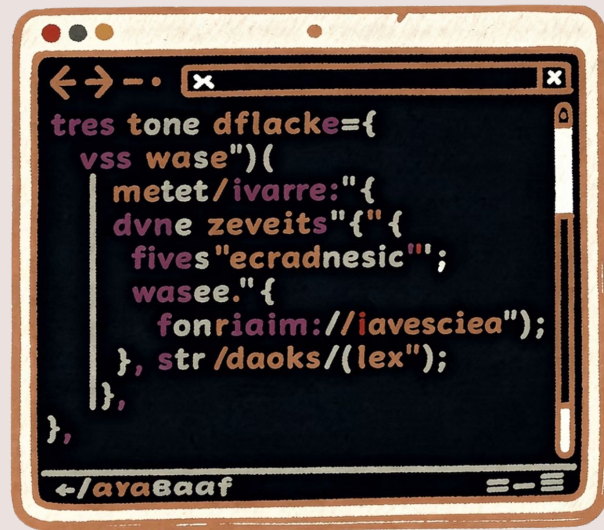
Preferência menor, não bloqueia. Indique explicitamente com *nit*: no comentário

❏ Classifique seus comentários para que o autor saiba o que é obrigatório corrigir e o que é apenas uma sugestão.



JavaScript Seguro na Prática

Ferramentas, padrões e exemplos reais
para um código JS confiável, legível e
manutenível.



ESLint + Prettier: Padronização Automática

Antes (sem padrão)

```
const x=getUserData()  
if(x!=null){  
doSomething(x)  
}
```

Depois (com ESLint + Prettier)

```
const userData = getUserData();  
if (userData !== null) {  
  doSomething(userData);  
}
```

Por que importa?

Padronização elimina discussões de estilo no review e mantém o foco no que realmente importa.

- **ESLint:** detecta erros e padrões inseguros
- **Prettier:** formata o código automaticamente
- Rodam no CI/CD antes do review humano

✅ Configure ambos no `.eslintrc` e `.prettierrc`, e adicione ao pipeline de CI.



Validação de Inputs: Protegendo a Lógica

❌ Código Vulnerável

```
function getUser(id) {  
  return db.query(  
    `SELECT * FROM users WHERE id = ${id}`  
  );  
}
```

✅ Código Seguro

```
function getUser(id) {  
  if (!Number.isInteger(id) || id <= 0) {  
    throw new Error('ID inválido');  
  }  
  return db.query(  
    'SELECT * FROM users WHERE id = ?',  
    [id]  
  );  
}
```

O que mudou?

- Validação explícita antes de usar o input
- Query parametrizada previne SQL Injection
- Erro semântico com mensagem clara

⊗ Nunca confie em dados externos sem validação. Inputs não tratados são a origem de boa parte das vulnerabilidades.



Código Limpo e Manutenível

✗ Difícil de revisar

```
function p(u, t) {  
  if (u && u.s === 1 && t) {  
    u.t = t;  
    save(u);  
  }  
}
```

Princípios aplicados

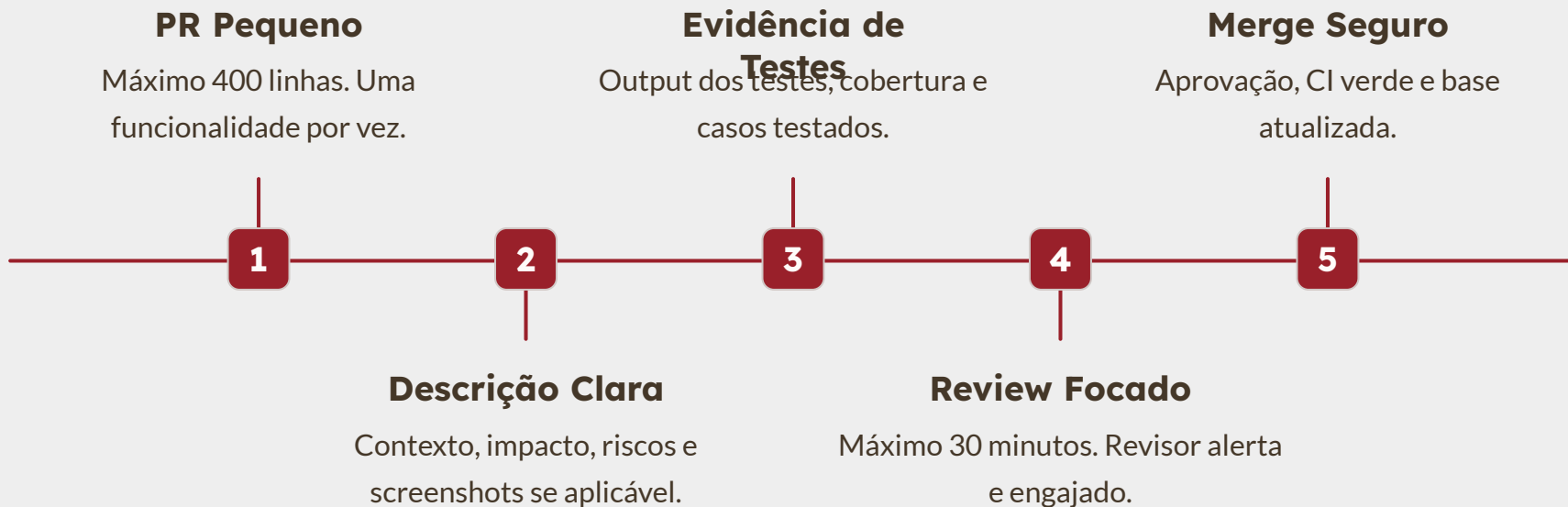
- **Nomes semânticos:** user, não u
- **Guard clause:** retorno antecipado reduz aninhamento
- **Variável explicativa:** isActiveUser documenta a intenção
- **Optional chaining:** acesso seguro a propriedades

✓ Fácil de revisar

```
function updateUserToken(user, token) {  
  const isActiveUser = user?.status ===  
  'active';  
  if (!isActiveUser || !token) return;  
  
  user.token = token;  
  saveUser(user);  
}
```



O Fluxo Ideal de um PR



PRs grandes são o inimigo número um da qualidade da revisão. Quanto maior o PR, menor a atenção por linha de código.



Checklist Avançado para o Projeto Final

Com a maturidade acumulada ao longo do semestre, chegou a hora de elevar o padrão de entrega. Não é só sobre código, é sobre o **sistema completo**.



Checklist: Legibilidade e Convenções

1

Nomes Semânticos

Variáveis e funções descrevem claramente sua intenção. Nenhuma abreviação obscura.

2

Princípio DRY

Lógica duplicada está extraída em funções reutilizáveis ou utilitários.

3

Estrutura Consistente

Padrão de organização de arquivos e pastas seguido em todo o projeto.

4

Linter Configurado

ESLint e Prettier configurados, sem warnings ignorados no CI.



Checklist: Testes

Unitários

Funções críticas
testadas isoladamente
com mocks

Integração

Fluxos principais
validados de ponta a
ponta

E2E

Jornadas do usuário cobertas em ambiente real

O que verificar no review

- Cobertura mínima definida e sendo respeitada
- Casos de sucesso e casos de falha cobertos
- Respostas HTTP mapeadas: 400, 404, 422, 500
- Testes legíveis com nomes descritivos

 Teste que não falha nunca pode estar testando a coisa errada.



Checklist: Segurança



Sem Credenciais no Código

Nenhuma senha, token ou chave de API no repositório. Use variáveis de ambiente e `.env`.



Autenticação e Autorização

Rotas protegidas, tokens com expiração e verificação de permissões por recurso.



Prevenção de Injeção

Queries parametrizadas, validação de schema e sanitização de inputs em todas as entradas externas.

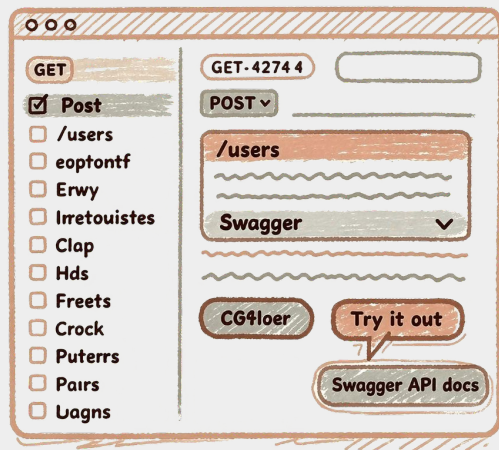


Erros sem Vazamento

Mensagens de erro genéricas para o cliente. Stack traces apenas em logs internos.



Checklist: Documentação



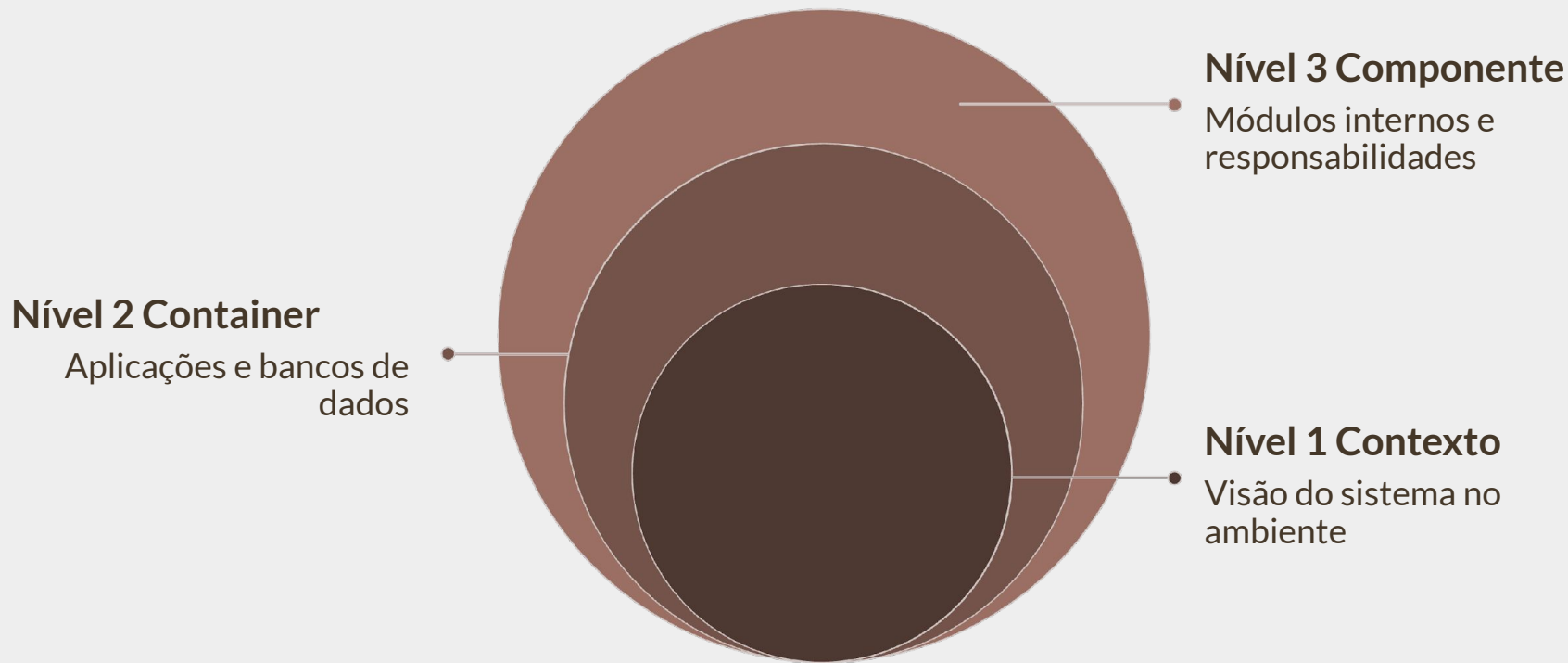
O que deve estar documentado

- **OpenAPI/Swagger:** todos os endpoints, parâmetros e respostas documentados
- **README:** instruções de setup, variáveis de ambiente e como rodar os testes
- **Comentários de código:** apenas onde a intenção não é óbvia pelo código em si
- **CHANGELOG:** histórico de decisões relevantes e mudanças de

- ☐ Documentação desatualizada é pior que nenhuma documentação, ela desinforma.



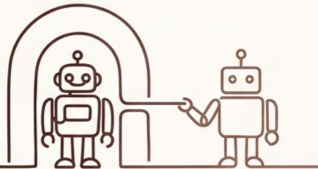
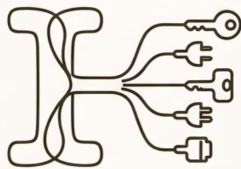
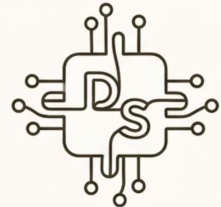
Checklist: Diagramas C4



O modelo C4 comunica a arquitetura em camadas de abstração progressiva. Para a entrega final, os níveis **1 e 2 são obrigatórios**. O nível 3 demonstra maturidade arquitetural.



Checklist: Arquitetura e SOLID

S - SINGLE RESPONSIBILITY 	O - OPEN/CLOSED 
S - Única Razão Para Mudar	O - Aberto Para Extensão
L - LISKOV SUBSTITUTION 	I - INTERFACE SEGREGATION 
L - Subtipos Substituem	I - Interfaces Específicas
D - DEPENDENCY INVERSION 	
D - Dependente De Abstrações 	

Perguntas para o review

- A classe ou módulo tem uma única responsabilidade?
- A mudança exigiu alterar múltiplos lugares não relacionados?
- As dependências estão invertidas via injeção?
- A arquitetura definida no início foi respeitada?

✓ Aderência ao SOLID é sinal de código que envelhece bem.

Checklist Completo: Visão Geral

Categoria	Critério Principal	Status
Legibilidade	Nomes semânticos, DRY, linter sem erros	☐ Verificado
Testes	Unitários, integração, E2E e casos de erro	☐ Verificado
Segurança	Sem credenciais, inputs validados, erros seguros	☐ Verificado
Documentação	OpenAPI atualizado, README completo	☐ Verificado
Arquitetura	SOLID, diagramas C4 (níveis 1 e 2)	☐ Verificado
Convenções	Padrão de projeto seguido integralmente	☐ Verificado



Estudo de Caso: o PR que Dois Revisores Aprovaram

O cenário

Equipe com processo rigoroso: todo PR precisa de **2 aprovações** antes do merge. Um PR aprovado por dois membros **derrubou o servidor de produção** na sexta à noite.

O que aconteceu

- O PR mudava uma função de validação chamada em **7 lugares diferentes**
- O review focou na lógica nova, ninguém verificou onde mais a função era usada
- Nenhum teste de regressão cobria os pontos

⚠ Review não é só verificar o que foi adicionado, é verificar o **impacto sistêmico** do que foi alterado. Dois aprovadores não garantem revisão sistêmica.



Do que Verificar ao Por que Verificar

Um checklist avançado vai além de "o código roda?". Ele olha para cinco dimensões que juntas garantem um software sustentável.



Funcionalidade

A user story está coberta? Casos de borda tratados? Erros explícitos ou silenciados?



Testes

Cobertura na lógica crítica? Testes legíveis? Teste passando ≠ funcionalidade funciona.



Legibilidade

Nomes comunicam intenção? Há duplicação? O código se explica sozinho?



Arquitetura

Separação de camadas respeitada? Lógica de negócio no lugar certo? Módulo sobrecarregado?



Documentação

Decisões registradas? README atualizado? Diagramas C4 refletem o estado atual?



Code Review é Comunicação — Não Sentença

A parte técnica do review todo mundo aprende. A parte que ninguém ensina é que **o jeito como você escreve um comentário determina se a pessoa vai crescer ou travar**. A diferença não é só no tom, é na ação que o comentário gera.

Reviews que destroem ❌

- "Isso está errado."
- "Código confuso."
- "Sem testes suficientes."

Diagnóstico sem direção. Gera defensividade, não evolução.

Reviews que constroem ✅

- "Consideraria usar X aqui, o que você acha?"
- "Esse trecho ficou difícil de seguir para mim, poderia renomear Y?"
- "Falta um teste para o caso em que o campo vem nulo."

Específico, acionável e respeitoso.



Os Benefícios que se Tornaram Tangíveis

Após semanas de Pull Requests e Code Reviews sistemáticos, a equipe da PetFood parou de apagar incêndios e começou a construir com confiança.



Rastreabilidade completa

Cada linha tem uma história. git blame revela o contexto de qualquer mudança, quem, quando e por quê.



Segurança na colaboração

Branches eliminaram o medo de sobrescrever o trabalho do colega. Cada um avança sem destruir o do outro.



Qualidade contínua

Nenhum código chega à main sem revisão e testes automáticos. A qualidade deixou de ser evento e virou processo.



"Dominar o controle de versão é instalar o alicerce sobre o qual toda a automação e agilidade contemporâneas são construídas." — Eng. Soft. em Dimensões, Cap. 2



Antes da Quinta — Olhem para Dentro

Na quinta-feira, o repositório de vocês será lido por outra equipe. Não como exercício — como avaliação real de legibilidade, qualidade e profissionalismo. Antes disso, respondam em equipe:

Pergunta 1

Qual é o pior trecho do código de vocês para outro desenvolvedor entender? Já existe um plano para melhorá-lo antes da quinta?

Pergunta 2

Os reviews que vocês fizeram nos últimos sprints usavam o checklist das 5 dimensões visto hoje? O que ficou de fora?

Pergunta 3

Se vocês fossem a equipe que vai revisar o código de vocês na quinta, o que esperariam encontrar, e o que encontrariam de verdade?

 Discussão em equipe: Seja honesto, não com o professor, com você mesmo.



Para Levar Além desta Aula

"Code review é uma prática técnica ou uma prática de cultura de equipe? Pode existir uma sem a outra?"

Um time pode ter checklist, ferramentas e CI configurado, e ainda assim ter uma cultura de review tóxica que paralisa quem está aprendendo. O inverso também é verdade: uma cultura de feedback generoso sem processo técnico sólido gera código gentil, mas frágil.

A questão não tem resposta certa, mas a sua resposta revela como você enxerga engenharia de software.

Cultura de equipe

Feedback construtivo e mentoria contínua.

Processo técnico

Checklist, CI e ferramentas padronizadas.

Produto sustentável

Qualidade, rastreabilidade e confiança.



Próximas 72 Horas

O semestre está na reta final. O que você faz nas próximas 72 horas vai determinar o estado do repositório que outra equipe vai ler na quinta.

1

Hoje

Revisem o repositório com o checklist das 5 dimensões. Identifiquem os pontos críticos antes que outra equipe os encontre.

2

Terça (amanhã)

Documentação, README e checklist de entrega, a outra face da qualidade final.
Leitura: Garcia, Cap. 19, seção 19.4.

3

Quinta (04/06)

Code review cruzado formal, o repositório de vocês será lido por outra equipe. Traga uma decisão técnica que nunca foi documentada.

⚠ Leitura para amanhã: Garcia, Cap. 19, seção 19.4, e traga uma decisão técnica do projeto que nunca foi documentada.



A Jornada do Profissional

Qualidade como Marca

O cuidado com o código reflete diretamente sua reputação como profissional

Checklist como Cultura

Uma ferramenta viva que evolui com o time e com o projeto

Entregue Sistemas

Não apenas linhas de código, entregue confiança, robustez e valor real

Software bem revisado não é sorte, é o resultado de disciplina, empatia e compromisso com a excelência.



O Melhor Review que Você Pode Dar...

"...é aquele que você gostaria de ter recebido."

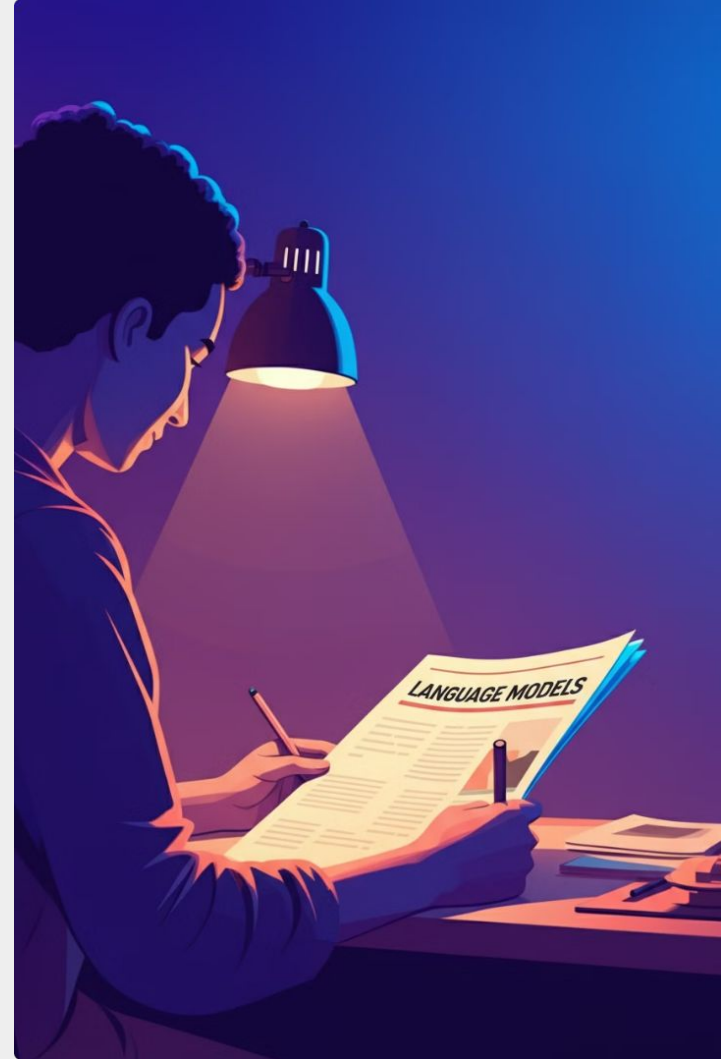
Revise com a generosidade que você esperaria de alguém mais experiente. Seja específico. Seja respeitoso. Ensine enquanto aponta. É assim que times ficam melhores, um PR de cada vez.

CIN0136 • CIn-UFPE • 2026.1 — Referências: Garcia, V. C. *Engenharia de Software em Dimensões*. ASSERT Lab, 2025. Cap. 2 (seção 2.5) e Cap. 3 (seção 3.5.1). Valente, M. T. *Engenharia de Software Moderna*. 2020. Cap. 10.



Próximos passos

- Eng. Software em Dimensões
 - Cap. 3 (seções 3.1–3.7): Versionamento, Git, PRs, Code Review, licenças
 - Cap. 14 (seções 14.1–14.8): Arquitetura, C4 Model, qualidade arquitetural
 - Cap. 19 (seções 19.1–19.4): Manutenção, refactoring, dívida técnica, documentação
- Eng. Software Moderna
 - Cap. 9 — RefactoringCatálogo, code smells, prática
 - Cap. 10 — DevOpsControle de versões, integração contínua



contacts

vcg@cin.ufpe.br
[@vinicius3w](https://www.instagram.com/vinicius3w)
viniciusgarcia.me

academy

go.uaite.xyz/vcg-es
go.uaite.xyz/vcg-microservices
go.uaite.xyz/vcg-devops
go.uaite.xyz/vcg-esaia
go.uaite.xyz/vcg-deso

Desenvolvimento de Software (CIN0136)

Code Review: Prática Profissional
e Segurança